

01_DIAGNÓSTICO — SA 01

Projeto SISGED — Sistema de Gestão Educacional Dinâmico

Squad Evora Dev — Ana Luiza Dutra Moreira (UI/UX) · Pedro Maia Fontes Lage (Front-end/JS) · Guilherme Duarte Assunção (Back-end/PHP) · Leonardo Fernandes de Carvalho (Banco de Dados) · Data: 11/09/2026

O sistema já tem uma base funcional declarada (autenticação e gestão de Turmas), mas a equipe ainda não concluiu a execução manual prevista na SA 01 (testar login, logout, cadastro, consulta, alteração e controle de acesso rodando localmente em WAMP/XAMPP). Os itens abaixo seguem exatamente a estrutura pedida no item 6 da SA 01 para o 01_DIAGNOSTICO.md; onde a informação ainda depende da execução em aula, isso está sinalizado.

1. Funcionalidades Existentes

- Autenticação (login e logout) com controle de sessão via PHP.
- Redirecionamento automático por sessão (index.php): usuário autenticado vai para turmas.html, não autenticado vai para login.html.
- Cadastro, consulta e alteração (CRUD) de Turmas.
- Conexão com banco de dados MySQL via PDO (app/config/database.php).
- Estrutura em MVC: app/config, app/controllers, app/models e public/views separados.
- Front-end em HTML5/CSS3 mobile-first, com JavaScript (Fetch API/AJAX) para comunicação com o back-end.

2. Funcionalidades Funcionando

- Roteamento inicial (index.php): confirmado por leitura de código — inicia sessão corretamente e redireciona para a view certa conforme login.
- Existência das views de login e de turmas (public/views/login.html e turmas.html), conforme estrutura do repositório.

As demais funcionalidades (login efetivo, CRUD de Turmas, controle de acesso) dependem da execução manual do sistema (Etapa 1 da SA 01) para serem confirmadas como funcionando — ainda não realizada pela equipe.

3. Funcionalidades com Problemas

- Login/logout: sem confirmação de teste real (sessão, expiração, redirecionamento em cenários de erro).
- CRUD de Turmas: sem confirmação de validação para campo vazio, dado inválido ou registro duplicado.
- Controle de acesso por perfil: sem confirmação de que perfis diferentes são de fato bloqueados nas ações restritas.
- Segurança do PDO: uso de prepared statements declarado no README, mas não confirmado nos arquivos reais de app/models.

4. Principais Falhas Encontradas

#	Falha / ponto a verificar em execução
1	Login, logout e controle de sessão: comportamento ainda não testado com o sistema rodando (WAMP/XAMPP) — validar redirecionamento e persistência da sessão além do que já está em index.php.
2	CRUD de Turmas (cadastrar, consultar, alterar): testar campo vazio, dado inválido e registro duplicado, conforme roteiro da SA 01.
3	Controle de acesso por perfil: confirmar se é possível acessar telas/ações fora do perfil autenticado.

#	Falha / ponto a verificar em execução
4	Prepared statements no PDO: o README declara uso, mas app/controllers e app/models precisam ser revisados para confirmar que não há SQL concatenado ou senha exposta.
5	Integridade do banco (schema.sql): checar chaves primárias/estrangeiras e se INSERT/UPDATE mantêm consistência entre as tabelas.
6	Organização MVC: confirmar se não há HTML/SQL misturado dentro dos controllers, já que public/views é anunciado como "views limpas sem PHP".

5. Classificação das Falhas

Falha	Gravidade	Motivo
Prepared statements / senha exposta não confirmados	Crítica	Risco de segurança (SQL injection, vazamento de credencial) se não confirmado
Controle de acesso por perfil não testado	Alta	Pode permitir acesso indevido a dados/ações de outro perfil
Validação de campo vazio/dado inválido no CRUD de Turmas	Alta	Compromete integridade dos dados cadastrados
Integridade referencial do schema.sql	Média	Pode gerar inconsistência entre tabelas relacionadas
Login/logout e sessão não testados end-to-end	Média	Base do sistema, mas já há lógica implementada em index.php
Organização MVC (SQL/HTML fora do lugar)	Baixa	Impacta manutenibilidade, não a operação imediata

6. Responsável por Cada Correção

Falha	Responsável	Observação
Login, logout e sessão	Guilherme (Back-end)	Responsável técnico por PHP/lógica de back-end no contrato.
CRUD de Turmas	Guilherme (Back-end)	Apoio de Pedro na integração front-end via Fetch/AJAX.
Controle de acesso por perfil	Guilherme (Back-end)	Regra de negócio no PHP; Ana Luiza valida na interface.
Prepared statements / segurança do PDO	Guilherme (Back-end)	Revisar app/config/database.php e app/models.
Integridade do banco (schema.sql)	Leonardo (Banco de Dados)	Responsável por modelagem de dados e MySQL no contrato.
Organização MVC / SQL fora de lugar	Guilherme e Leonardo	Back-end e banco revisam juntos controllers e models.

Falha	Responsável	Observação
Integração tela → JS → PHP → banco	Pedro (Front-end/JS)	Responsável por JavaScript e integração front-end no contrato.
Interface e documentação das evidências	Ana Luiza (UI/UX)	Responsável por UI/UX e HTML/CSS; registra prints e evidências.

7. Definição do MVP que Será Validado

- ☒ Autenticação (login/logout) com controle de sessão
- ☒ Cadastro, consulta e alteração (CRUD) de Turmas
- ☒ Controle de acesso por perfil de usuário
- ☒ Persistência das operações no banco (INSERT/SELECT/UPDATE) com integridade referencial
- ☐ Módulos adicionais (Instrutores, Salas, Agendamentos) além do já existente no repositório
- ☐ Novas funcionalidades ou melhorias visuais antes da conclusão deste diagnóstico